Disciplina

Teste de Software

Detecção de Bad Smells e Refatoração Segura

Relatório do Trabalho Prático

Aluno: Arlindo Sérgio Pereira Junior

Matrícula: 856882

Repositório: github.com/ArlindoSPJr/bad-smells-js-refactoring

Outubro de 2026

1. Análise de Smells

O código analisado é um serviço que gera relatórios de itens nos formatos CSV e HTML, filtrando o conteúdo de acordo com o perfil do usuário. Toda a lógica está concentrada em um único método, *generateReport*. A leitura cuidadosa do código revelou três Bad Smells principais.

1.1 Método Longo (Long Method)

O método principal tem 58 linhas e acumula seis responsabilidades: montar o cabeçalho, aplicar a regra de visibilidade por perfil, marcar os itens prioritários, formatar cada linha, somar o total e montar o rodapé. Os próprios comentários que dividem o método em “cabeçalho”, “corpo” e “rodapé” indicam que ele deveria ser dividido.

Impacto na manutenção: qualquer alteração, por menor que seja, exige compreender o método inteiro, e o risco de afetar outra parte sem perceber é alto.

Impacto nos testes: não é possível testar isoladamente a regra de visibilidade, o cálculo do total ou a formatação de uma linha. Os testes só conseguem verificar o texto final completo, o que torna as falhas mais difíceis de localizar.

1.2 Condicionais Complexas e Aninhadas

O corpo do relatório é um laço com até quatro níveis de decisão encadeados: o perfil do usuário, a faixa de valor do item e o formato do relatório. Além disso, o formato é verificado em quatro pontos diferentes do método: no cabeçalho, em cada um dos dois ramos de perfil e no rodapé.

Impacto na manutenção: incluir um novo formato, como JSON, ou um novo perfil obriga a alterar vários trechos ao mesmo tempo, e é fácil esquecer algum deles.

Impacto nos testes: cada combinação de perfil, formato e faixa de valor é um caminho de execução diferente. O número de cenários necessários para cobrir o método cresce de forma multiplicativa.

1.3 Código Duplicado (Duplicated Code)

A linha em CSV é escrita de forma idêntica para os dois perfis, a linha em HTML é quase idêntica, e a soma do total aparece repetida quatro vezes.

Impacto na manutenção: uma mudança no formato da linha, como acrescentar uma coluna, precisa ser feita em mais de um lugar. Esquecer uma das cópias gera relatórios inconsistentes entre os perfis.

Impacto nos testes: cada cópia é um ponto de falha independente e precisa de cobertura própria. Um teste que passa para um perfil não garante o comportamento do outro.

Também foram observados números mágicos (os limites 500 e 1000 sem nome que explique seu significado), a alteração dos dados recebidos pelo método (o item é marcado como prioritário no próprio objeto de entrada) e uma dependência de banco de dados recebida no construtor, mas nunca utilizada.

2. Relatório da Ferramenta

A análise estática foi feita com o ESLint e o plugin *eslint-plugin-sonarjs*, usando as regras recomendadas do SonarJS, limite de complexidade cognitiva igual a 15 e a regra de detecção de funções idênticas. A Figura 1 mostra o resultado obtido antes da refatoração.

```
$ npx eslint src/

C:\Faculdade\Testes\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Figura 1 – Resultado da análise estática antes da refatoração.

A ferramenta apontou dois erros:

- **Complexidade cognitiva 27**, quase o dobro do limite de 15. A análise manual já indicava que o método era complexo, mas a ferramenta transformou essa percepção em um número objetivo e verificável, que serve de régua para medir a melhoria depois da refatoração.

- **Condicional colapsável**: uma decisão aninhada que poderia ser unida à condição externa. Esse ponto passou despercebido na leitura manual, o que mostra a vantagem de uma verificação sistemática, que não se cansa nem se distrai.

Por outro lado, a ferramenta não detectou a duplicação quase idêntica, a alteração dos dados de entrada nem a dependência sem uso. Esses são problemas de design, que exigem compreender a intenção do código. Uma execução complementar com regras nativas do ESLint confirmou ainda o tamanho excessivo do método (58 linhas e 32 instruções) e os números mágicos 500 e 1000. A conclusão é que a análise manual e a ferramenta se complementam: a ferramenta é rápida, objetiva e repetível, enquanto a análise humana enxerga os problemas de projeto.

3. Processo de Refatoração

O smell mais crítico era o das condicionais complexas e aninhadas, responsável pela complexidade cognitiva 27. Ele está diretamente ligado aos outros dois: é o aninhamento que torna o método longo e que obriga a repetir a formatação em cada ramo. Por isso, corrigi-lo resolveu os três problemas ao mesmo tempo.

Antes – corpo do relatório:

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
      }
    }
  }
}
```

Depois – regras de visibilidade, formatador e método principal:

```
const visibilityRules = {
  [Roles.ADMIN]: () => true,
  [Roles.USER]: (item) => item.value <= USER_MAX_VISIBLE_VALUE,
};

class CsvFormatter {
  header() { return 'ID,NOME,VALOR,USUARIO\n'; }
  row(item, user) { return `${item.id},${item.name},${item.value},${user.name}\n`; }
  footer(total) { return `\nTotal,,\n${total},,\n`; }
}
// HtmlFormatter implementa a mesma interface: header, row e footer

generateReport(reportType, user, items) {
  const formatter = formatters[reportType];
  if (!formatter) {
    return '';
  }

  const visibleItems = items.filter((item) => canView(user, item));
  const body = visibleItems.map((item) => formatter.row(item, user)).join('');
  const total = calculateTotal(visibleItems);

  return (formatter.header(user) + body + formatter.footer(total)).trim();
}
```

Técnicas aplicadas:

- **Replace Conditional with Polymorphism:** cada formato passou a ser uma classe (CSV e HTML) com a mesma interface: cabeçalho, linha e rodapé. O formato é verificado uma única vez, na escolha do formatador. Um novo formato exige apenas uma nova classe, sem alterar o método principal.
- **Decompose Conditional:** a regra que define o que cada perfil pode ver saiu do laço e virou uma tabela de regras por perfil, consultada por uma função própria. Os quatro níveis de decisão deram lugar a um filtro seguido de uma transformação.
- **Extract Method:** o cálculo do total, a regra de prioridade e a regra de visibilidade viraram funções pequenas, cada uma com um único propósito. A soma, antes repetida quatro vezes, passou a ser feita uma única vez.
- **Replace Magic Number with Symbolic Constant:** os limites 500 e 1000 e os perfis de usuário receberam nomes que explicam seu significado.

Além disso, a prioridade passou a ser calculada sem alterar os itens recebidos, e a dependência de banco de dados sem uso foi removida. A tabela abaixo resume o resultado.

Métrica	Antes	Depois
Erros apontados pelo SonarJS	2	0
Complexidade cognitiva (maior função)	27	1
Linhas do método principal	58	12
Pontos que verificam o formato do relatório	4	1
Testes aprovados	5 de 5	5 de 5

Tabela 1 – Comparação entre o código original e o refatorado.

A suíte de testes original foi duplicada, alterando apenas a referência ao código testado, e executada após cada passo da refatoração. Os dez testes (cinco da versão original e cinco da refatorada) passaram durante todo o processo. As duas versões também foram comparadas em 18 combinações de formato, perfil e valores limite, e produziram saídas idênticas. Após a refatoração, a análise estática não apontou nenhum problema.

4. Conclusão

Refatorar é mudar a estrutura do código sem mudar o seu comportamento. A suíte de testes foi a garantia dessa segunda parte: como os testes verificam o resultado final, e não a forma como ele é produzido, foi possível reorganizar o código por completo com segurança. Cada passo era seguido da execução dos testes, e qualquer regressão seria percebida imediatamente, quando ainda é fácil de localizar e desfazer. Sem essa rede de segurança, cada mudança dependeria de confiança, e não de evidência.

A redução dos Bad Smells trouxe ganhos concretos de qualidade. O código ficou mais legível, porque cada parte tem um nome que descreve sua intenção; mais fácil de manter, porque cada regra está em um único lugar; e mais fácil de estender, porque um novo formato de relatório não exige alterar o código existente. Também ficou mais testável, já que as regras de negócio agora são funções pequenas e isoladas. Por fim, o trabalho mostrou que ferramentas de análise estática e revisão humana se complementam: a ferramenta mede e aponta, e o desenvolvedor interpreta e decide como melhorar o design.